Yelamos_Sanz_Blanca-PEC1

Análisis de datos Ómicos - Primera prueba de evaluación continua

Tabla de contenidos

Ejercicio 1. Exploración multivariante de un dataset de metabolómica

1. Introducción y objetivos
2. Métodos
3. Resultados
4. Discusión
5. Referencias
6. Apéndice con el código utilizado

Ejercicio 2. Revisión de un artículo

Ejercicio 1. Exploración multivariante de un dataset de metabolómica

1. Introducción y objetivos

En este ejercicio se va a trabajar con unos datos de metabolómica proporcionados, y el objetivo que se persigue es establecer un flujo de análisis exploratorio sobre este conjunto de datos. Se pretende que se sea capaz de construir un análisis razonado y organizado trabajando con los contenedores ómicos introducidos en Bioconductor, explorar datos ómicos multivariantes, aplicar técnicas de reducción de dimensión y discutir de manera crítica los resultados obtenidos y la utilidad de los análisis empleados.

Este documento se organiza en varias partes:

- una primera con este informe sobre los datos de metabolómica que a su vez consta de una estructura en: introducción y objetivos, métodos, resutlados, discusión, referencias y apéndice con el código.
- y una segunda parte que consta del segundo ejercicio de la prueba de evaluación, que consiste en la revisión de un artículo.

2. Métodos

Los datos con los que se van a trabajar son datos de metabolómica proporcionados en tres archivos .csv en el aula virtual. Los tres archivos proporcionados contienen la matriz de intensidades metabolómicas (assay.csv), la información fenotípica y técnica de las muestras (sample_metadata.csv) y la anotación de los metabolitos medidos (feature_metadata.csv).

Se estableció un flujo de análisis exploratorio con diferentes herramientas que irán mencionando y que se llevó a cabo en el entorno de RStudio de la forma en que se detalla en el apéndice.

Una vez cargados los datos se procedió a la creación de un contenedor de la clase SummerizedExperiment (como indica el enunciado). Esta clase permite integrar la matriz de datos y metadatos de muestras y metabolitos, manteniendo alineadas las dimensiones de la matriz con los metadatos, evitando errores al filtrar, reordenar o eliminar muestras o metabolitos en el proceso de preparación de los datos previo al análisis.

Este contenedor es una clase más reciente que ExpressionSet que tiene su uso orientado principalmente a worflows de microarrays, mientras que SummerizedExperiment permite la inclusión de múltiples assays y tiene una estructura de datos más estandarizada y compatible con otros paquetes de Bioconductor, facilitando pasos posteriores en el análisis de datos como PCA o clustering.

El dataset original contiene 60 muestras y un total de 120 metabolitos. Los metadatos sobre las muestras incluyen información sobre el grupo experimental (caso o control), el lote, el sexo, la edad, el índice de masa corporal y los niveles de colesterol de cada muestra. Mientras que los metadatos de las variables contienen información sobre el identificador del metabolito, la clase química y la tecnología empleada para medir ese metabolito (platform).

Se generó un resumen descriptivo de las intensidades por muestra mediante summary(). Las distribuciones mostraron rangos amplios y diferencias entre muestras, lo cual es habitual en datos metabolómicos sin transformar. No se observaron valores extremos evidentes, aunque este análisis motivó la posterior transformación logarítmica para estabilizar la varianza.

El enunciado solicita trabajar con un subconjunto de 50 muestras, manteniendo un reparto equilibrado entre los dos grupos principales. Para ello realicé un muestreo aleatorio estratificado para ganrantizar representatividad equilibrada. Se fija una semilla (set.seed(1234)) para asegurar que el muestreo sea reproducible, y se seleccionan 25 muestras de cada grupo.

El filtrado se realiza sobre el objeto SummarizedExperiment garantizando que la matriz de intensidades y los metadatos se mantienen alineados, y generando un nuevo contenedor (se_sub) que será el que se utilice en adelante.

Posteriormente a obtener el contenedor definitivo con los datos, hay que hacer una preparación de estos, un preprocesamiento antes de llevar a cabo el análisis.

En primer lugar, examiné la presencia de valores perdidos por metabolito. Siguiendo la recomendación habitual en metabolómica y el criterio indicado en el enunciado, eliminé aquellos metabolitos con más del 20% de valores faltantes, ya que un alto nivel de estos valores puede indicar problemas en la detección e introducir ruido en análisis posteriores.

Para los metabolitos con un porcentaje inferior al 20%, imputé los valores faltantes mediante el algoritmo KNN (k-Nearest Neighbors). Este método identifica, para cada metabolito con valores perdidos, los k metabolitos más similares segú su patrón de intensidades, y utiliza esos valores para estimar el dato que falta. Esto preserva la estructura multivariante del dataset y evita eliminar información útil cuando estamos antes pocos NA.

La imputación se realizó utilizando la función impute.knn() del paquete impute de Bioconductor. De esta manera se consiguieron unos datos sin valroes perdidos.

Los datos metabolómicos suelen presentar distribuciones muy asimétricas, con valores extremos y rangos amplios. Para estabilizar la varianza y reducir la influencia de valores muy altos, apliqué una transformación logarítmica en base 2. Esta transformación reduce la asimetría, hace las variables más comparables y facilita la interpretación del PCA. Normalmente se evitan problemas con ceros al sumar 1 antes del logaritmo, ese paso en este caso no ha sido necesario dado que todos los valores son positivos y mayores de 0.

Tras la imputación y la transformación logarítmica, evalué la presencia de valores atípicos mediante la regla de Tukey, aplicada de forma independiente a cada metabolito. Para ello, calculé el primer y tercer cuartil (Q1 y Q3) de cada fila de la matriz log-transformada, obtuve el rango intercuartílico (IQR = Q3 − Q1) y definí los límites inferior y superior. Este procedimiento permite identificar valores puntuales anómalos dentro de cada metabolito.

En total se identificaron 44 valores atípicos, frente a 5956 valores no atípicos, lo que representa un porcentaje muy reducido del total. Este resultado es coherente con la transformación logarítmica aplicada previamente, que reduce la asimetría y la presencia de valores extremos, y con la imputación KNN, que tiende a suavizar valores anómalos aislados. Esto hace que los cuartiles sean más representativos y que la detección basada en rango intercuatílico (IQR, regla de Tukey) sea más fiable.

El análisis por metabolito y por muestra mostró que los outliers están distribuidos de forma dispersa y no concentrados en una muestra concreta.

Además de la detección de valores atípicos por metabolito, evalué la posible presencia de muestras outlier (con patrón de internsidades muy diferente del resto) mediante un boxplot de las intensidades ya transformadas logarítmicamente, dado que reduce la asimetría típica de los datos metabolómicos y permite comparar de forma más fiable las distribuciones entre muestras. Para ello representé la distribución de valores de cada muestra.

En el gráfico con los boxplot de cada muestra no se observaron muestras con medianas desplazadas, rangos intercuartílicos anómalos ni un número elevado de valores extremos. Esto sugiere que ninguna muestra presenta un comportamiento global claramente atípico.

Con los datos ya preparados procedí a llevar a cabo el Análisis de Componentes Principales (PCA), que se aplicó al dataset previamente filtrado, imputado y transformado logarítmicamente. Antes de realizar el PCA, dentro de la misma función se centraron y escalaron las variables (en lugar de hacerlo en pasos previos), con el fin de estandarizar los datos y de esta forma que todas las variables contribuyan de forma equilibrada al análisis, independientemente de su escala original, y así observar la correlación entre variables.

Una vez obtenido el PCA se mostró mediante gráficos la contribución de los componentes principales a la varibilidad del sistema, la relación con otra variables fenotípicas como el sexo y la edad, y la contribución de los 10 metabolitos más influyentes en los dos primeros componentes principales.

Con los datos del PCA se generaron los resultados del análisis que se van a comentar a continuación.

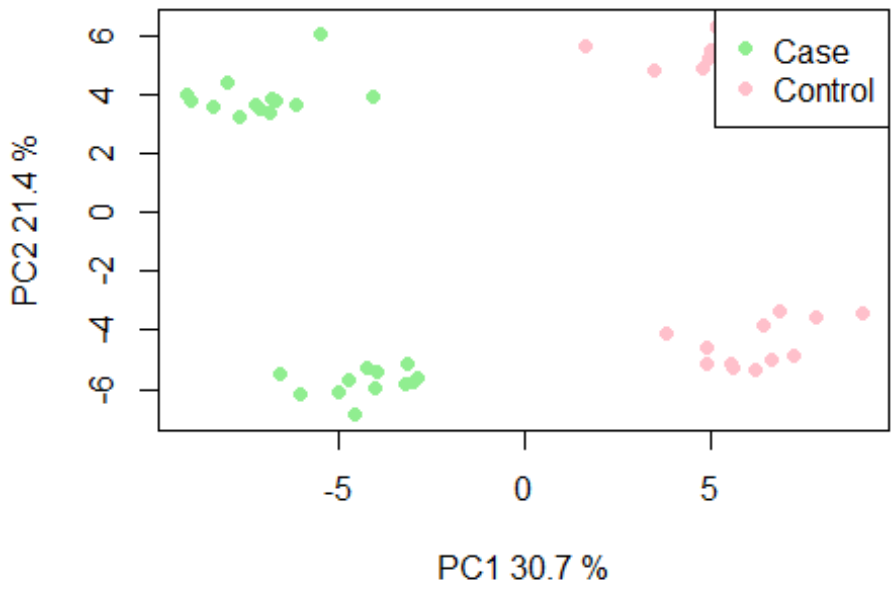
3. Resultados

Los resultados obtenidos del PCA fueron los siguientes: se calculó la variabilidad explicada por cada componente y observamos que los 4 primeros explican respectivamente el 30.7%, 21.4%, 8.3% y 6.3% de los datos. Esto nos podría indicar que retuviesemos y explorásemos los primeros componentes principales.

A modo descriptivo y dado que explican en conjunto el 52.1% de la variabilidad de los datos metabolómicos, en este caso se va a llevar a cabo la representación de los 2 primeros componentes principales (PC1 frente a PC2) y vamos a explorar el resultado de este gráfico y la relación con las variables de las muestras, y la contribución de los metabolitos a estos los primeros 2 componentes.

```
loads <- round(pca$sdev^2 / sum(pca$sdev^2)*100, 1)

# Creamos las etiquetas para los ejes del gráfico PC1vsPC2
xlab = c(paste("PC1", loads[1], "%"))
ylab = c(paste("PC2", loads[2], "%"))
```

```
# Extraemos los grupos experimentales: casos y controles
group = colData(se_clean)$group

# Colores por grupo
color <- ifelse(group == "Case", "lightgreen", "pink")

# Gráfico PCA
plot(pca$x[,1], pca$x[,2], col = color, pch = 19, xlab = xlab, ylab =
ylab)
legend("topright", legend=c("Case","Control"),
col=c("lightgreen","pink"), pch=19)
```



En el gráfico se observan las muestras distribuidas según los dos primeros componentes principales con una clara separación en agrupamientos. Cada punto del gráfico es una muestra y su posición refleja su perfil metabolómico en base a PC1 y PC2.

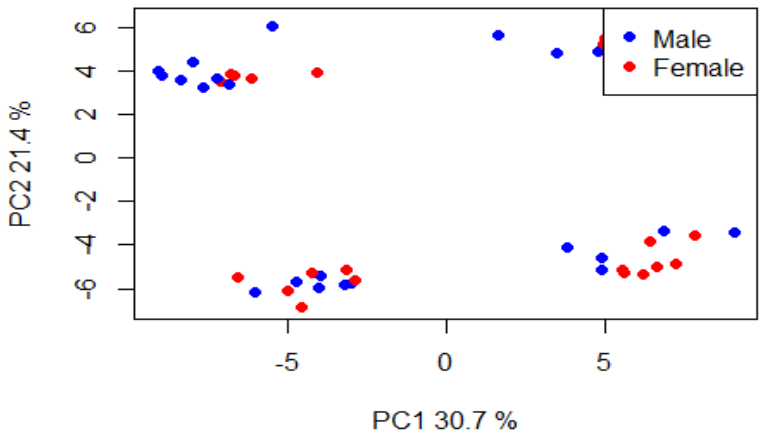
Lo primero que se puede observar en el gráfico es el porcentaje de variabilidad explicado por cada componente: 30.7% en el caso del PC1, y 21.4% del PC2, que explica la segunda mayor parte de variabilidad independiente de PC1. Juntos explican más del 50% de la variación total, lo cual es bastante informativo en metabolómica.

El grupo de los casos, representados en color verde, se agrupan en la parte izquierda del gráfico, mientras que los controles, en rosa, se agrupan en la parte derecha. Esto indica que los perfiles de ambos grupos son distintos y sugiere que hay metabolitos o conjuntos de ellos que diferencian ambos grupos.

El PC1 distingue de manera clara ambos grupos, lo que indica que la condición biológica (caso o control) es una de las principales fuentes de variación en el perfil metabolómico de los datos. El hecho de que el PCA sea un método no supervisado hace esta separación muy relevante, dado que la agrupación de las muestras procede únicamente de las diferencias en los metabolitos y no de la etiqueta de grupo.

Para completar el análisis se evaluó si PC1 o PC2 se relacionaban con otras variables fenotípicas. En este caso por ejemplo, se va a estudiar a modo ilustrativo la relación con el sexo como variable categórica, y la edad como variable numérica. Se analizará el efecto de la variable categórica del sexo mediante la creación de un gráfico de los 2 componentes principales coloreando las muestras por sexo. Mientras que para la variable numérica se van a calculas los coeficientes de correlación de esta variable con ambos componentes.

```
# Gráfico PC1vsPC2 muestras estratificadas por sexo
sex <- colData(se_clean)$sex
color_sex <- ifelse(sex == "M", "blue", "red")

plot(pca$x[,1], pca$x[,2], col = color_sex, pch = 19, xlab = xlab, ylab =
ylab)
legend("topright", legend=c("Male","Female"), col=c("blue","red"),
pch=19)
```



En el caso del sexo se muestra una distribución heterogénea de las muestras y no se observan patrones claros asociados a esta variable.

La edad muestra una correlación negativa moderada con PC1 (-0.55), lo que sugiere que parte de la variabilidad de este componente puede estaría asociada a diferencias relacionadas con la edad. A mayor edad, los valores de PC1 tienden a ser menores, lo cual podría indicar un patrón metabólico relacionado con envejecimiento.

La correlación de la edad con PC2 fue débil (-0.19), lo cual indica que este componente no está muy influido por la variable edad.

Se quiso analizar también los metabolitos que más contribuyen a PC1 y PC2. Se pueden observar los metabolitos más influyentes en el PC1, que son los que más contribuyen a esa separación entre casos y controles (M025 y M040 son los primeros). Estos metabolitos podrían representar rutas metabólicas alteradas entre ambos grupos y constituir candidatos para análisis posteriores.

También se observarn los metabolitos más influyentes en el PC2 (M001 y M002 los primeros), que podrían estar relacionados con la estructura secundaria que se observaba en el eje vertical del gráfico de los componentes principales.

4. Discusión

El análisis exploratorio realizado permite obtener una visión integrada de la estructura general del dataset metabolómico que se nos daba en el ejercicio, y de los factores que influyen en su variabilidad. Tras el filtrado de metabolitos con alto porcentaje de valores perdidos, la imputación mediante el algoritmo KNN y la transformación logarítmica, los datos quedaron preparados y en condiciones adecuadas para aplicar el PCA.

El PCA mostró que la mayor parte de la variabilidad se concentraba en las primeras componentes principales (PC1 y PC2), que explican de manera conjunta más del 50 % de la variación total. La representación del gráfico PC1 vs PC2 mostró una separación clara a lo largo de PC1 entre los grupos experimentales de casos y controles. Esto indica que hay diferencias importantes entre los perfiles metabolómicos de ambos grupos, lo cual sugiere la presencia de algún patrón biológico.

Además, la edad mostró una correlación moderada con PC1, lo que sugiere que, a pesar de que no parezca ser el factor principal, podría tener cierto efecto en la separación entre grupos. Al contrario que el sexo, que no parece mostrar un patrón claro ni una correlación importante con ninguno de los dos primeros componentes principales del análisis.

Aunque el PCA ha sido útil para entender la estructura general de los datos, también tiene sus limitaciones: puede dejar fuera relaciones más complejas entre metabolitos, y aunque vemos una separación clara entre grupos, no hemos comprobado la significación estadística, ni identificado formalmente qué metabolitos cambian entre ellos. Por todo esto, sería necesaria una segunda fase de análisis más exhaustivo, aplicar métodos más complejos y que no se han probado para ver si aparecen otras estructuras, analizar metabolitos que se han expresado de manera diferencial entre grupos para detectar posibles biomarcadores, o explorar rutas metabólicas asociadas a los metabolitos más influyentes de esos primeros componentes.

En conjunto, creo que el análisis exploratorio y el PCA han permitido visualizar la estructura global del dataset, identificar una separación clara entre casos y controles, descartar muestras atípicas, evaluar la influencia de algunas variables fenotípicas y determinar qué metabolitos contribuyen más a la variabilidad principal. Por tanto, considero que este análisis proporciona una primera aproximación a los patrones que caracterizan a los datos metabolómicos de este estudio.

5. Referencias

- Bioconductor. (s. f.). impute: Imputation for microarray data. Recuperado de https://bioconductor.org/packages/release/bioc/html/impute.html

- Center for Cancer Research Bioinformatics Training. (s. f.). Lesson 7: Introduction to Bioconductor -omics classes (containers). Recuperado el 31 de marzo de 2026 de https://bioinformatics.ccr.cancer.gov/docs/data-wrangle-with-r/Bioconductor_L7/

- El Mundo de los Datos. (s. f.). Análisis de componentes principales (PCA): simplificando datos complejos. Recuperado de https://elmundodelosdatos.com/analisis-de-componentes-principales-pca-simplificando-datos-complejos/

- Morgan, M., Obenchain, V., Hester, J., & Pagès, H. (2025). SummarizedExperiment: A container (S4 class) for matrix-like assays (R package version 1.40.0). Bioconductor. https://bioconductor.org/packages/SummarizedExperiment https://doi.org/10.18129/B9.bioc.SummarizedExperiment

- NursingHero. (s. f.). Study files. Recuperado de https://www.nursinghero.com/study-files/24835217

- PlotNerd. (s. f.). Método de Tukey vs MAD para detección de outliers. Recuperado de https://plotnerd.com/es/blog/mad-vs-tukey-outlier-detection/#tukey-method

- van den Berg, R. A., Hoefsloot, H. C., Westerhuis, J. A., Smilde, A. K., & van der Werf, M. J. (2006). Centering, scaling, and transformations: improving the biological information content of metabolomics data. BMC Genomics, 7(1), 142. https://doi.org/10.1186/1471-2164-7-142

- Wei, R., Wang, J., Su, M. et al. Missing Value Imputation Approach for Mass Spectrometry-based Metabolomics Data. Sci Rep 8, 663 (2018). https://doi.org/10.1038/s41598-017-19120-0

6. Apéndice con el código

```
# Vamos a cargar los datos desde los archivos csv
assay <- read.csv("PEC1_dataset/assay.csv", row.names = 1)
sample_metadata <- read.csv("PEC1_dataset/sample_metadata.csv", row.names
= 1)
feature_metadata <- read.csv("PEC1_dataset/feature_metadata.csv",
row.names = 1)
```

El argumento row.names = 1 permite que la primera columna se interprete como los nombres de las filas, paso imprescindible para mantener la correspondencia entre la matriz y los metadatos.

Antes de construir el contenedor, compruebo que los archivos están alineados correctamente y que no hay duplicados.

```
# Comprobamos que los nombres de las columnas de la matriz son los mismos
que
# las muestras (filas) en sample_metadata
all(colnames(assay) == rownames(sample_metadata))

## [1] TRUE

# Comprobamos que los nombres de las filas de la matriz (metabolitos)
coinciden
# con los metabolitos (filas) en feature data
all(rownames(assay) == rownames(feature_metadata))

## [1] TRUE

# Comprobamos que no hay duplicados
anyDuplicated(colnames(assay))

## [1] 0

anyDuplicated(rownames(assay))

## [1] 0

# Construimos el contenedor SummerizedExperiment con los datos de
metabolómica
# como se indicaba en el enunciado
library(SummarizedExperiment)

## Cargando paquete requerido: MatrixGenerics

## Cargando paquete requerido: matrixStats

##
## Adjuntando el paquete: 'MatrixGenerics'

## The following objects are masked from 'package:matrixStats':
##
```

```
##     colAlls, colAnyNAs, colAnys, colAvgsPerRowSet, colCollapse,
##     colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
##     colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
##     colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
##     colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
##     colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
##     colWeightedMeans, colWeightedMedians, colWeightedSds,
##     colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgsPerColSet,
##     rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
##     rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
##     rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
##     rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
##     rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
##     rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
##     rowWeightedSds, rowWeightedVars

## Cargando paquete requerido: GenomicRanges

## Cargando paquete requerido: stats4

## Cargando paquete requerido: BiocGenerics

## Cargando paquete requerido: generics

##
## Adjuntando el paquete: 'generics'

## The following objects are masked from 'package:base':
##
##     as.difftime, as.factor, as.ordered, intersect, is.element,
setdiff,
##     setequal, union

##
## Adjuntando el paquete: 'BiocGenerics'

## The following objects are masked from 'package:stats':
##
##     IQR, mad, sd, var, xtabs

## The following objects are masked from 'package:base':
##
##     anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##     colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##     get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
##     order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
##     rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
##     unsplit, which.max, which.min

## Cargando paquete requerido: S4Vectors
```

```
##
## Adjuntando el paquete: 'S4Vectors'

## The following object is masked from 'package:utils':
##
##     findMatches

## The following objects are masked from 'package:base':
##
##     expand.grid, I, unname

## Cargando paquete requerido: IRanges

##
## Adjuntando el paquete: 'IRanges'

## The following object is masked from 'package:grDevices':
##
##     windows

## Cargando paquete requerido: Seqinfo

## Cargando paquete requerido: Biobase

## Welcome to Bioconductor
##
##     Vignettes contain introductory material; view with
##     'browseVignettes()'. To cite Bioconductor, see
##     'citation("Biobase")', and for packages 'citation("pkgname")'.

##
## Adjuntando el paquete: 'Biobase'

## The following object is masked from 'package:MatrixGenerics':
##
##     rowMedians

## The following objects are masked from 'package:matrixStats':
##
##     anyMissing, rowMedians

sumexp <- SummarizedExperiment(
  assays = list(intensity = as.matrix(assay)),
  colData = sample_metadata,
  rowData = feature_metadata
)
sumexp

## class: SummarizedExperiment
## dim: 120 60
## metadata(0):
## assays(1): intensity
## rownames(120): M001 M002 ... M119 M120
```

```
## rowData names(3): metabolite_name class platform
## colnames(60): S039 S022 ... S017 S026
## colData names(6): group batch ... bmi cholesterol
```

```r
# Vamos a ver y describir la estructura del dataset
dim(assay)
```

```
## [1] 120  60
```

```r
summary(assay)
```

```
##       S039             S022            S044             S043
##  Min.   : 372.2   Min.   :  300   Min.   : 292.7   Min.   : 372.6
##  1st Qu.: 993.6   1st Qu.:  712   1st Qu.: 853.9   1st Qu.:1064.8
##  Median :1963.5   Median : 1416   Median :1913.4   Median :2121.3
##  Mean   :2255.5   Mean   : 2119   Mean   :2321.4   Mean   :2244.1
##  3rd Qu.:3221.6   3rd Qu.: 2193   3rd Qu.:3310.0   3rd Qu.:2985.8
##  Max.   :6196.7   Max.   :10332   Max.   :8847.1   Max.   :7171.9
##  NA's   :11       NA's   :9       NA's   :2        NA's   :6
##       S020             S041             S010             S047
##  Min.   : 386.0   Min.   : 277.4   Min.   : 245.1   Min.   :  555.1
##  1st Qu.: 836.9   1st Qu.: 769.9   1st Qu.: 533.6   1st Qu.: 1862.7
##  Median :1739.3   Median :1543.0   Median : 945.0   Median : 3441.8
##  Mean   :2217.4   Mean   :1951.4   Mean   :1161.9   Mean   : 3955.3
##  3rd Qu.:3101.7   3rd Qu.:2661.1   3rd Qu.:1713.6   3rd Qu.: 5388.5
##  Max.   :6955.7   Max.   :7164.4   Max.   :4818.5   Max.   :14887.4
##  NA's   :11       NA's   :12       NA's   :8        NA's   :8
##       S003             S048              S055             S006
##  Min.   : 297.1   Min.   :  568.6   Min.   : 690.1   Min.   : 209.9
##  1st Qu.: 621.0   1st Qu.: 1638.9   1st Qu.:1365.9   1st Qu.: 545.1
##  Median : 906.8   Median : 3020.6   Median :2204.0   Median :1125.0
##  Mean   :1174.4   Mean   : 3657.9   Mean   :2876.5   Mean   :1464.9
##  3rd Qu.:1566.7   3rd Qu.: 5443.3   3rd Qu.:3633.7   3rd Qu.:1952.1
##  Max.   :4110.1   Max.   :12480.3   Max.   :9769.6   Max.   :6443.5
##  NA's   :4        NA's   :7         NA's   :6        NA's   :4
##       S053              S023              S059             S009
##  Min.   :  660.1   Min.   :  265.4   Min.   : 332.8   Min.   : 277.0
##  1st Qu.: 1865.0   1st Qu.:  697.3   1st Qu.:1389.5   1st Qu.: 518.0
##  Median : 3453.6   Median : 1671.8   Median :2631.9   Median : 736.4
##  Mean   : 4222.5   Mean   : 2250.9   Mean   :3250.3   Mean   : 996.3
##  3rd Qu.: 5736.3   3rd Qu.: 3070.5   3rd Qu.:4883.9   3rd Qu.:1358.4
##  Max.   :13409.2   Max.   :12269.7   Max.   :9432.8   Max.   :3104.3
##  NA's   :10        NA's   :5         NA's   :4        NA's   :5
##       S060             S027             S052              S015
##  Min.   : 360.7   Min.   : 287.7   Min.   :  718.1   Min.   : 215.2
##  1st Qu.:1674.1   1st Qu.: 641.4   1st Qu.: 1790.0   1st Qu.: 613.0
##  Median :3067.1   Median :1268.9   Median : 3124.0   Median : 829.4
##  Mean   :3336.3   Mean   :1820.8   Mean   : 3919.7   Mean   :1127.1
##  3rd Qu.:4493.5   3rd Qu.:2078.4   3rd Qu.: 4979.7   3rd Qu.:1394.4
##  Max.   :9115.2   Max.   :8845.7   Max.   :13079.5   Max.   :5371.7
##  NA's   :8        NA's   :3        NA's   :8         NA's   :6
```

```
##       S042             S040              S031             S032
##  Min.   : 330.9   Min.   :  337.7   Min.   : 331.7   Min.   : 295.7
##  1st Qu.:1050.9   1st Qu.:  997.5   1st Qu.: 734.6   1st Qu.:1190.2
##  Median :1889.2   Median : 1892.2   Median :1361.1   Median :2293.2
##  Mean   :2087.6   Mean   : 2147.6   Mean   :1998.1   Mean   :2599.7
##  3rd Qu.:2989.8   3rd Qu.: 2869.8   3rd Qu.:2967.1   3rd Qu.:3728.5
##  Max.   :6510.8   Max.   :10197.0   Max.   :6481.7   Max.   :8174.6
##  NA's   :10       NA's   :3         NA's   :1        NA's   :6
##       S024             S036             S016             S001
##  Min.   : 560.5   Min.   : 379.7   Min.   : 244.8   Min.   : 300.0
##  1st Qu.:1087.5   1st Qu.: 817.4   1st Qu.: 587.3   1st Qu.: 554.8
##  Median :1671.0   Median :1842.8   Median :1306.2   Median : 778.4
##  Mean   :2336.1   Mean   :1888.0   Mean   :2036.7   Mean   :1131.4
##  3rd Qu.:2908.2   3rd Qu.:2556.8   3rd Qu.:2407.3   3rd Qu.:1551.0
##  Max.   :8595.9   Max.   :5304.1   Max.   :8623.2   Max.   :3958.2
##  NA's   :7        NA's   :8        NA's   :8        NA's   :12
##       S030             S051              S012              S007
##  Min.   : 335.3   Min.   :  260.9   Min.   :  342.8   Min.   : 322.6
##  1st Qu.: 855.9   1st Qu.: 1531.1   1st Qu.:  725.5   1st Qu.: 541.6
##  Median :1725.3   Median : 2241.3   Median : 1117.3   Median : 853.3
##  Mean   :2335.8   Mean   : 2795.5   Mean   : 1805.6   Mean   :1102.2
##  3rd Qu.:3189.3   3rd Qu.: 3314.4   3rd Qu.: 1970.7   3rd Qu.:1637.2
##  Max.   :8418.4   Max.   :10239.5   Max.   :24805.0   Max.   :2962.5
##  NA's   :6        NA's   :7         NA's   :2         NA's   :8
##       S004             S037             S050              S058
##  Min.   : 246.3   Min.   : 270.4   Min.   :  493.9   Min.   :  554.4
##  1st Qu.: 568.3   1st Qu.:1065.7   1st Qu.: 1825.0   1st Qu.: 1629.9
##  Median : 892.1   Median :1994.6   Median : 3225.8   Median : 2471.4
##  Mean   :1294.9   Mean   :2390.4   Mean   : 3866.3   Mean   : 3299.2
##  3rd Qu.:1977.6   3rd Qu.:3536.5   3rd Qu.: 5262.3   3rd Qu.: 4958.3
##  Max.   :7318.2   Max.   :7891.1   Max.   :14442.4   Max.   :11572.7
##  NA's   :7        NA's   :3        NA's   :8         NA's   :9
##       S057             S021              S038              S011
##  Min.   : 754.8   Min.   :  268.8   Min.   :  330.8   Min.   : 250.5
##  1st Qu.:1748.9   1st Qu.:  738.9   1st Qu.:  956.5   1st Qu.: 558.1
##  Median :2804.1   Median : 1438.8   Median : 1867.8   Median :1001.8
##  Mean   :3376.1   Mean   : 2133.2   Mean   : 2763.4   Mean   :1366.4
##  3rd Qu.:4393.0   3rd Qu.: 2255.7   3rd Qu.: 4027.5   3rd Qu.:1698.9
##  Max.   :9287.7   Max.   :10133.9   Max.   :10267.0   Max.   :7311.3
##  NA's   :9        NA's   :9         NA's   :3         NA's   :6
##       S002             S054              S035             S005
##  Min.   : 291.6   Min.   :  308.3   Min.   : 221.7   Min.   : 207.5
##  1st Qu.: 556.3   1st Qu.: 1693.6   1st Qu.: 749.8   1st Qu.: 470.3
##  Median : 832.6   Median : 3080.0   Median :1651.7   Median : 816.4
##  Mean   :1099.9   Mean   : 3584.5   Mean   :2325.3   Mean   :1232.0
##  3rd Qu.:1536.8   3rd Qu.: 4889.6   3rd Qu.:3435.5   3rd Qu.:1733.1
##  Max.   :3099.1   Max.   :10875.2   Max.   :7925.9   Max.   :5969.3
##  NA's   :7        NA's   :7         NA's   :7        NA's   :6
##       S045             S049              S033             S029
##  Min.   : 240.6   Min.   :  606.6   Min.   : 256.1   Min.   :  203.8
```

```
##  1st Qu.:1007.2   1st Qu.: 1619.6   1st Qu.:1034.0   1st Qu.:  583.7
##  Median :1956.5   Median : 2940.7   Median :1921.7   Median : 1539.6
##  Mean   :2032.9   Mean   : 3530.0   Mean   :2251.5   Mean   : 2029.9
##  3rd Qu.:2835.5   3rd Qu.: 4516.3   3rd Qu.:3340.1   3rd Qu.: 2483.5
##  Max.   :7482.2   Max.   :12788.0   Max.   :6522.9   Max.   :10460.4
##  NA's   :4        NA's   :3         NA's   :8        NA's   :5
##       S046              S025             S019              S008
##  Min.   :  661.1   Min.   : 291.7   Min.   :  290.5   Min.   : 300.3
##  1st Qu.: 1574.7   1st Qu.: 673.3   1st Qu.:  698.2   1st Qu.: 660.5
##  Median : 2579.1   Median :1441.4   Median : 1530.3   Median :1139.8
##  Mean   : 3461.3   Mean   :2270.6   Mean   : 2341.5   Mean   :1378.1
##  3rd Qu.: 4889.7   3rd Qu.:3388.9   3rd Qu.: 2626.7   3rd Qu.:1770.4
##  Max.   :11102.2   Max.   :9558.0   Max.   :11069.7   Max.   :4439.7
##  NA's   :5         NA's   :9        NA's   :6         NA's   :9
##       S034             S013             S014             S056
##  Min.   : 243.1   Min.   : 258.4   Min.   : 310.8   Min.   : 589.5
##  1st Qu.:1060.7   1st Qu.: 503.8   1st Qu.: 626.8   1st Qu.:1759.0
##  Median :1892.6   Median : 899.2   Median : 959.1   Median :3591.9
##  Mean   :2083.4   Mean   :1171.7   Mean   :1189.3   Mean   :3540.2
##  3rd Qu.:2919.0   3rd Qu.:1717.9   3rd Qu.:1587.9   3rd Qu.:4619.6
##  Max.   :6941.8   Max.   :3988.3   Max.   :4255.7   Max.   :9362.2
##  NA's   :6        NA's   :11       NA's   :10       NA's   :10
##       S018              S028             S017             S026
##  Min.   :  354.5   Min.   : 280.2   Min.   : 276.1   Min.   :  312.7
##  1st Qu.:  776.4   1st Qu.: 617.1   1st Qu.: 599.3   1st Qu.:  665.0
##  Median : 1459.8   Median :1288.3   Median :1151.7   Median : 1509.6
##  Mean   : 2455.0   Mean   :1856.9   Mean   :1863.0   Mean   : 2132.2
##  3rd Qu.: 3024.8   3rd Qu.:1951.9   3rd Qu.:2292.7   3rd Qu.: 2295.3
##  Max.   :12675.0   Max.   :9950.1   Max.   :7533.1   Max.   :11829.4
##  NA's   :4         NA's   :2        NA's   :6        NA's   :9
```

```r
head(sample_metadata)
```

```
##        group  batch sex age  bmi cholesterol
## S039    Case Batch1   M  44 27.4       222.6
## S022 Control Batch2   F  47 21.0       157.7
## S044    Case Batch1   F  52 28.2       206.8
## S043    Case Batch1   M  58 28.5       212.7
## S020 Control Batch2   M  52 22.0       168.4
## S041    Case Batch1   F  45 27.4       236.8
```

```r
head(feature_metadata)
```

```
##      metabolite_name         class       platform
## M001  Metab_batch_01    Amino acid LC-MS positive
## M002  Metab_batch_02 Acylcarnitine LC-MS negative
## M003  Metab_batch_03     Bile acid          GC-MS
## M004  Metab_batch_04    Fatty acid LC-MS positive
## M005  Metab_batch_05         Lipid LC-MS negative
## M006  Metab_batch_06  Organic acid          GC-MS
```

```r
# Vamos a crear el subdataset de 50 muestras que se va a utilizar
posteriormente
# Comprobamos los grupos que tenemos y su reparto equilibrado: casos y
controles
table(sample_metadata$group)

##
##    Case Control
##      30      30

# Fijamos una semilla para reproducibilidad
set.seed(1234)

# Cargamos la librería tidyverse para el filtro y selección de los grupos
library(tidyverse)

## ── Attaching core tidyverse packages ────────────────────────
tidyverse 2.0.0 ──
## ✓ dplyr     1.2.0     ✓ readr     2.2.0
## ✓ forcats   1.0.1     ✓ stringr   1.6.0
## ✓ ggplot2   4.0.2     ✓ tibble    3.3.1
## ✓ lubridate 1.9.5     ✓ tidyr     1.3.2
## ✓ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────
tidyverse_conflicts() ──
## ✗ lubridate::%within%() masks IRanges::%within%()
## ✗ dplyr::collapse()     masks IRanges::collapse()
## ✗ dplyr::combine()      masks Biobase::combine(),
BiocGenerics::combine()
## ✗ dplyr::count()        masks matrixStats::count()
## ✗ dplyr::desc()         masks IRanges::desc()
## ✗ tidyr::expand()       masks S4Vectors::expand()
## ✗ dplyr::filter()       masks stats::filter()
## ✗ dplyr::first()        masks S4Vectors::first()
## ✗ dplyr::lag()          masks stats::lag()
## ✗ ggplot2::Position()   masks BiocGenerics::Position(),
base::Position()
## ✗ purrr::reduce()       masks GenomicRanges::reduce(),
IRanges::reduce()
## ✗ dplyr::rename()       masks S4Vectors::rename()
## ✗ lubridate::second()   masks S4Vectors::second()
## ✗ lubridate::second<-() masks S4Vectors::second<-()
## ✗ dplyr::slice()        masks IRanges::slice()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force
all conflicts to become errors

# Filtramos, extraemos y guardamos los nombres de las muestras que
pertencecen
```

```r
# a los casos del sample_metadata
grupo1 <- rownames(sample_metadata %>% filter(group == "Case"))
# Del mismo modo que antes, pero en este caso se filtran, extraen y
guardan los
# nombres de los controles
grupo2 <- rownames(sample_metadata %>% filter(group == "Control"))

# Elegimos 25 muestras aleatorias de cada grupo, y se combinan en un
vector
sel1 <- sample(grupo1, 25)
sel2 <- sample(grupo2, 25)
muestras_seleccionadas <- c(sel1, sel2)

# Filtramos el SummarizedExperiment original y guardamos uno nuevo con la
# selección de 50 muestras aleatorias
# Le decimos que coja todas las filas (metabolitos) y las columnas de
# "muestras seleccionadas"
se_sub <- sumexp[, muestras_seleccionadas]
se_sub
```

```
## class: SummarizedExperiment
## dim: 120 50
## metadata(0):
## assays(1): intensity
## rownames(120): M001 M002 ... M119 M120
## rowData names(3): metabolite_name class platform
## colnames(50): S046 S036 ... S019 S002
## colData names(6): group batch ... bmi cholesterol
```

```r
library(SummarizedExperiment)
library(impute) # previamente instalada
# Examinamos la presencia de valores perdidos en las variables, se extrae
el
# assay a una matriz y se examinan los valores perdidos
mat <- assay(se_sub, "intensity")
na_prop <- rowMeans(is.na(mat))

# Decimos que nos quedamos con los metabolitos cuyo porcentaje de NA <=
0.20
keep <- na_prop <= 0.20

# Filtramos la matriz para quedarnos con los metabolitos aceptables
# Es decir, con las filas donde se cumple que keep es TRUE
mat_filt <- mat[keep, ]

# Y para esa matriz de valores que nos queda con % NA's <= 0.20 se
imputan
# los valores con el algoritmo KNN
mat_imputed <- impute.knn(mat_filt)$data
```

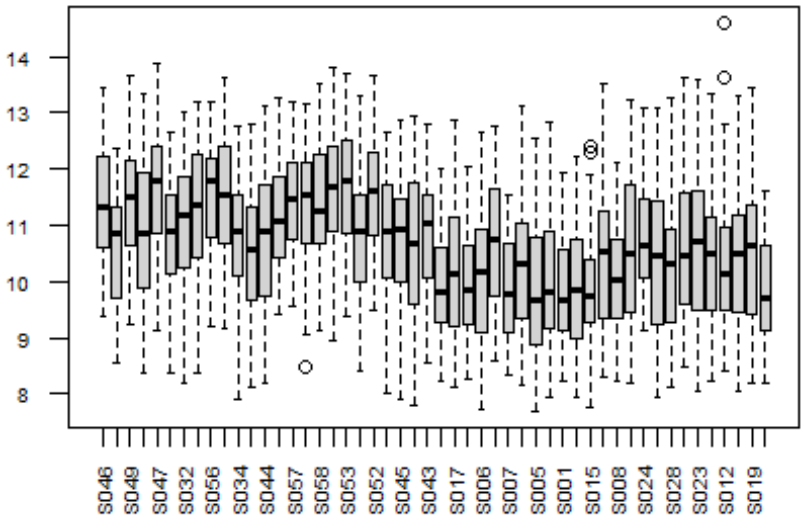
```r
# Filtramos el objeto se_sub para conservar los metabolitos que pasaron
el filtro
se_clean <- se_sub[keep, ]

# Reemplazamos el assay por la matriz con los datos imputados
assay(se_clean, "intensity") <- mat_imputed

# Vamos a realizar una transformación logarítmica a los datos de la
matriz
# Hacemos el logaritmo en base 2 de la matriz de datos
mat_log <- log2(mat_imputed)

# Y reemplazamos esta nueva matriz logarítmica en el objeto se_clean
assay(se_clean, "log_intensity") <- mat_log

# Aplicamos la regla de Tukey para ver valores outliers con el IQR
# Calculamos los cuartiles y el IQR
Q1 <- apply(mat_log, 1, quantile, 0.25)
Q3 <- apply(mat_log, 1, quantile, 0.75)
IQR <- Q3 - Q1

# Definimos los umbrales inferior y superior
lower <- Q1 - 1.5 * IQR
upper <- Q3 + 1.5 * IQR

# Generamos una matriz lógica que compara los elementos de la matriz
mat_log
# con los vectores lower y upper
outlier_matrix <- (mat_log < lower) | (mat_log > upper)

# Miramos la cantidad de valores outliers detectados y los que no lo son
sum(outlier_matrix)

## [1] 44

sum(!outlier_matrix)

## [1] 5956

# Miramos los outliers por metabilito y por muestra
rowSums(outlier_matrix)

## M001 M002 M003 M004 M005 M006 M007 M008 M009 M010 M011 M012 M013 M014
M015 M016
##    0    0    0    0    0    0    0    0    0    0    0    0    0    0
0    0
## M017 M018 M019 M020 M021 M022 M023 M024 M025 M026 M027 M028 M029 M030
M031 M032
##    0    0    0    0    0    0    0    0    0    0    0    0    0    0
0    0
## M033 M034 M035 M036 M037 M038 M039 M040 M041 M042 M043 M044 M045 M046
```

```
M047 M048
##    0    0    0    0    0    0    0    0    0    0    0    0    0    0
0    0
## M049 M050 M051 M052 M053 M054 M055 M056 M057 M058 M059 M060 M061 M062
M063 M064
##    0    0    0    0    0    0    0    0    0    0    0    0    1    2
1    0
## M065 M066 M067 M068 M069 M070 M071 M072 M073 M074 M075 M076 M077 M078
M079 M080
##    0    2    2    2    1    0    0    1    0    1    4    0    2    0
0    0
## M081 M082 M083 M084 M085 M086 M087 M088 M089 M090 M091 M092 M093 M094
M095 M096
##    0    0    1    0    0    0    0    0    0    0    0    0    0    0
3    0
## M097 M098 M099 M100 M101 M102 M103 M104 M105 M106 M107 M108 M109 M110
M111 M112
##    0    1    0    0    0    0    4    2    0    1    0    2    0    0
0    4
## M113 M114 M115 M116 M117 M118 M119 M120
##    0    0    7    0    0    0    0    0
```

```r
colSums(outlier_matrix)
```

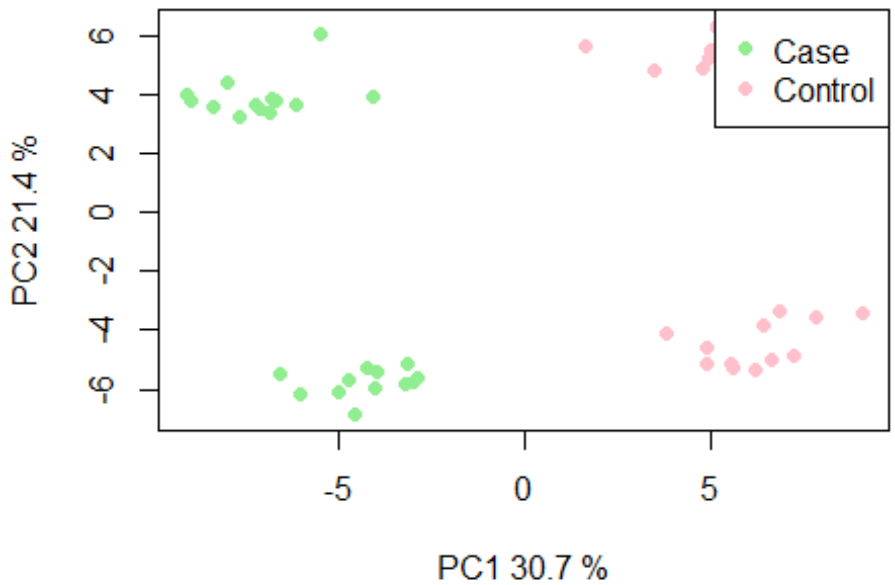
```
## S046 S036 S049 S038 S047 S042 S032 S059 S056 S048 S034 S041 S044 S055
S057 S060
##    0    2    2    2    4    0    1    1    0    0    1    1    0    2
0    1
## S058 S050 S053 S040 S052 S033 S045 S035 S043 S003 S017 S014 S006 S020
S007 S027
##    0    0    0    0    1    1    1    0    1    1    5    0    0    0
0    1
## S005 S004 S001 S010 S015 S026 S008 S025 S024 S016 S028 S018 S023 S022
S012 S021
##    2    2    0    1    2    2    0    0    0    1    0    1    1    0
3    0
## S019 S002
##    1    0
```

```r
boxplot(mat_log, las = 2, cex.axis = 0.6)
```

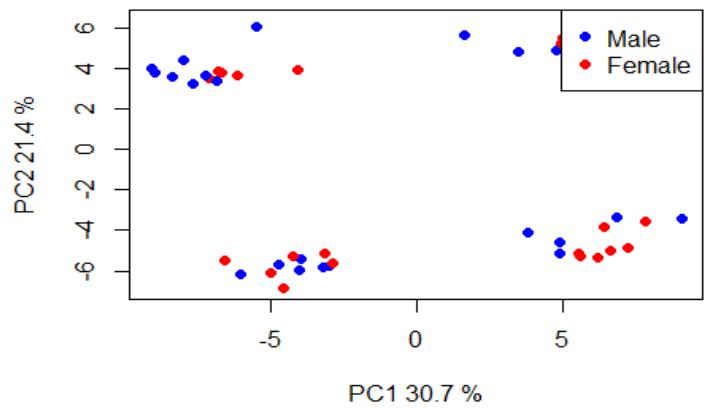



```
pca <- prcomp(t(mat_log), center = TRUE, scale. = TRUE)

# Guardamos en la variable loads la variabilidad explicada por cada
componente
loads <- round(pca$sdev^2 / sum(pca$sdev^2)*100, 1)

# Mostramos la variabilidad que explica cada componente
loads

##  [1] 30.7 21.4  8.3  6.3  2.6  2.3  2.2  2.1  1.8  1.7  1.7  1.5  1.4
1.3  1.2
## [16]  1.1  1.0  0.9  0.9  0.8  0.8  0.8  0.7  0.7  0.6  0.6  0.6  0.5
0.4  0.3
## [31]  0.3  0.3  0.3  0.3  0.2  0.2  0.2  0.2  0.1  0.1  0.1  0.1  0.1
0.1  0.1
## [46]  0.1  0.0  0.0  0.0  0.0

# Creamos las etiquetas para los ejes del gráfico PC1vsPC2
xlab = c(paste("PC1", loads[1], "%"))
ylab = c(paste("PC2", loads[2], "%"))

# Extraemos los grupos experimentales: casos y controles
group = colData(se_clean)$group

# Colores por grupo
color <- ifelse(group == "Case", "lightgreen", "pink")
```

```r
# Gráfico PCA
plot(pca$x[,1], pca$x[,2], col = color, pch = 19, xlab = xlab, ylab =
ylab)
legend("topright", legend=c("Case","Control"),
col=c("lightgreen","pink"), pch=19)
```




```r
# Gráfico PC1vsPC2 muestras estratificadas por sexo
sex <- colData(se_clean)$sex
color_sex <- ifelse(sex == "M", "blue", "red")

plot(pca$x[,1], pca$x[,2], col = color_sex, pch = 19, xlab = xlab, ylab =
ylab)
legend("topright", legend=c("Male","Female"), col=c("blue","red"),
pch=19)
```




```
# Calculamos la correlación de la edad con PC1 y PC2
age <- colData(se_clean)$age
cor_PC1_age <- cor(pca$x[,1], age)
cor_PC2_age <- cor(pca$x[,2], age)
cor_PC1_age

## [1] -0.546953

cor_PC2_age

## [1] -0.1866308

# Se cargan en loadings la contribución de los metabolitos a cada
componente
loadings <- pca$rotation

# Los 10 metabolitos más influyentes en PC1
head(sort(abs(loadings[,1]), decreasing = TRUE), 10)

##      M025      M040      M046      M047      M038      M039      M026
M022
## 0.1622613 0.1618220 0.1615223 0.1612548 0.1611902 0.1611654 0.1611065
0.1610998
##      M049      M034
## 0.1610834 0.1610830

# Los 10 metabolitos más influyentes en PC2
head(sort(abs(loadings[,2]), decreasing = TRUE), 10)

##      M001      M002      M016      M015      M018      M007      M017
M014
## 0.1939962 0.1931734 0.1930652 0.1929025 0.1925948 0.1924799 0.1923103
0.1922786
##      M006      M003
## 0.1922489 0.1922407
```

Ejercicio 2. Revisión de un artículo

El artículo seleccionado para llevar a cabo este segundo ejercicio de la Prueba de Evaluación Continua 1 se titula “Single-cell RNA-seq reveals new types of human blood dendritic cells, monocytes and progenitors”, cuya referencia completa en estilo APA7 es: Villani, A.-C., Satija, R., Reynolds, G., Sarkizova, S., Shekhar, K., Fletcher, J., Griesbeck, M., Butler, A., Zheng, S., Lazo, S., Jardine, L., Dixon, D., Stephenson, E., Nilsson, E., Grundberg, I., McDonald, D., Filby, A., Li, W., De Jager, P. L., ... Hacohen, N. (2017). Single-cell RNA-seq reveals new types of human blood dendritic cells, monocytes and progenitors. Science (New York, N.Y.), 356(6335), eaah4573. https://doi.org/10.1126/science.aah4573.

En este artículo se analizan datos de transcriptómica de célula única (single cell RNA, scRNA-seq) de células inmunes humanas, específicamente de células dendríticas y monocitos obtenidas de sangre periférica. Esto genera un dataset muy grande y típico de las tecnologías ómicas, con muchas variables, correlaciones y ruido.

La pregunta general del estudio es identificar y caracterizar estos 2 tipos celulares del sistema inmune humano, ya que debido a la escasez de marcadores para células dendríticas puede afectar a la asignación de funciones y la ontogenia de cada subtipo. En el artículo se quiere observar la diversidad real y abrir la posibilidad a una clasificación más refinada que la que había en el momento presente del artículo (2017) de estos tipos celulares, con las posibles repercusiones a nivel de funcionalidad y ontogenia que puede conllevar, incluyendo también poblaciones o subtipos nuevos, o progenitores circulantes, no descritos hasta el momento o mal definidos. Para lo cual caracterizan transcriptómicamente las células.

En el contexto de este estudio, los autores aplican el Ánalisis de Componentes Principales (PCA) porque el scRNA-seq genera datos de miles de genes por célula, un espacio de datos tan grande y con alta dimensionalidad que resulta complejo de analizar, visualizar e interpretar la estructura directamente. Por ello, el PCA permite resumir la variación dominante y observar si las células se agrupan de forma natural, es decir, capturar la mayor parte de la variabilidad en menos variables (componentes principales), lo cual ayuda a eleminar ruido ténico y detectar efectos de batch, además de facilitar la visualización e interpretación de patrones en los datos. Utilizan el PCA como herramienta exploratoria, esa reducción de dimensión permite extraer las características principales que permiten clasificar y agrupar las células en subtipos basados en su similitud transcriptómica.

Para llevar a cabo el análisis, los autores seleccionaron los genes más variables del dataset, y encontraron 595 genes con alta variabilidad entre células. Entonces aplicaron el PCA para reducir la dimensionalidad de los datos, y evaluaron los componentes principales con un test de permutación para seleccionar los 5 más significativos.

Esos primeros componentes se utilizaron como input para la técnica de t-distributed Stochastic Neighbor Embedding (t-SNE) y, posteriormente, se llevó a cabo la técnica de clustering para agrupar las células en 6 clústeres de células dendríticas (DC1 a DC6) y 4 clústeres de monocitos (Mono1 a Mono4), es decir, se utiizó el PCA como herramienta preliminar del análisis de datos.

Luego, validaron los agrupamientos utilizando marcadores diferenciales de superficie para aislar los subtipos identificados y llevaron a cabo perfiles scRNA-seq adicionales para confirmar la clasificación.

En cuanto a la interpretación de los resultados, el PCA les ayudó a descubrir una variación biológica real, la estrcutura de los datos y orientar el análisis hacia poblaciones celulares relevantes. Junto con los métodos posteriores de análisis ya mencionados, los autores descubre 6 subtipos de células dendríticas en sangre humana, incluyendo uno de ellos que muestra características intermedias entre pDCs y otras DCs (AS DCs = DC5), además de identificar 4 subtipos de monocitos, ampliando la clasificación clásica y mostrando una heterogeneidad superior a la que se conocía en el momento. Con ello también descubrieron un progenitor circulante de células dendríticas convencionales y muestran que la diferenciación transcriptómica se correlaciona con funciones inmunológicas más específicas y la posible relevancia de ello en enfermedades.

Los autores concluyen que la reducción dimensional y el clustering les ha permitido redefinir la heterogeneidad y desarrollo, la clasificación previa de células dendríticas y monocitos en sangre humana, lo cual ofrece una base para futuros estudios funcionales y clínicos.

Considero que la reducción de dimensiones con el PCA está bien elegida, fundamentada y ejecutada en este artículo, siendo un componente importante para responder las preguntas biológicas, organizando los datos y revelenado una estructura biológica de los mismos. El uso del PCA es muy adecuado para abordar el alto volumen y complejidad del dataset transcriptómico y apropiado para empezar a explorar los datos, reduciendo esa dimenisonalidad, detectando patrones globales e identificando subpoblaciones, y eliminando ruido y preparando los datos para análisis posteriores más complejos. Con el empleo del PCA seguido de t-SNE y clustering, permite la visualización, división y exploración de subpoblaciones celulares.

Además los autores van más allá, no quedándose solo en el análisis bioinformático, sino utilizando los resultados para seleccionar marcadores de superficie para separar físicamente las subpoblaciones y realizar validaciones funcionales y transcriptómicas, fortaleciendo la rubustez del método empleado y su utilidad. Si bien es cierto, que admiten que la transcriptómica tiene limitaciones intrínsecas, por ejemplo, no identificar células con baja expresión de ARNm, y por ello en futuras investigaciones se deberían integrar datos multi-ómicos y otras condiciones tanto físicas como biológicas.

En conclusión, en cuanto al uso del PCA y la aplicación de la reducción de la dimensionalidad creo que es adecuada, que está bien justificada e integrada en el flujo de análisis de los datos del artículo.

Enlace a repositorio GitHub:

https://github.com/blancayelamos03-code/Yelamos_Sanz_Blanca-PEC1.git